

Adaptive Multi-Cell Battery Intelligence Engine

Unified Battery Management System — Wokwi Simulation Project

Submitted by: Krutika Yogesh Malkhede

Internship Program: ElevanceSkil

Project Type: Embedded Systems & IoT — Battery Management System

Platform: Wokwi Simulator (ESP32) + Blynk IoT Cloud

Project Link: wokwi.com/projects/467059005329336321

This report documents the design, implementation, and integration of all six challenge tasks into a single, unified Wokwi project, demonstrating a complete, production-style Battery Management System (BMS) for a simulated 4-cell lithium battery pack.

1. Project Overview

All six challenge tasks were integrated and demonstrated within a single unified Wokwi project, in line with the assessment requirement that separate task-wise submissions would not be evaluated individually. The system simulates a 4-cell lithium battery pack using potentiometers as analog cell-voltage sources, processed by an ESP32 microcontroller, and presents real-time diagnostics on a 20x4 I2C LCD, LED/buzzer indicators, a relay-based protection circuit, and a cloud-connected Blynk dashboard.

The implementation strictly follows an event-driven, non-blocking architecture using **millis()**-based timing throughout — no blocking **delay()** calls are used in the main control loop — ensuring the system remains responsive to real-time sensor changes, protection triggers, and cloud communication simultaneously.

1.1 Hardware Components Used

Component	Quantity	Purpose
ESP32 Dev Module	1	Main microcontroller; runs all logic, WiFi, and Blynk communication
Potentiometer	4	Simulates individual lithium cell voltages (mapped to 2.5V–4.2V)
20x4 I2C LCD Display	1	Rotating multi-screen Human-Machine Interface (HMI)
LED (Green)	1	Indicates HEALTHY battery status
LED (Yellow)	1	Indicates MINOR IMBALANCE / FLUCTUATION status
LED (Red)	1	Indicates CRITICAL IMBALANCE / OVERVOLTAGE / SENSOR ERROR
LED (Blue)	1	Indicates PACK FAILURE (weak cell) status
Buzzer	1	Non-blocking audible alert during fault conditions
Relay Module	1	Simulated load/charge cutoff for safety protection

No additional hardware was required for Tasks 3 through 6, as the ESP32's built-in WiFi module and the existing LCD/LED/relay circuit were sufficient to fulfil the HMI, fault-tolerance, telemetry, and dashboard requirements purely through firmware logic.

2. Task 1 — Adaptive Multi-Cell Battery Intelligence Engine

A modular battery intelligence engine was implemented to monitor four simulated lithium cells in real time. Analog readings from four potentiometers are mapped from the raw 0–3.3V ADC range into a realistic lithium-cell voltage range of 2.5V–4.2V, ensuring all health states are reachable through normal potentiometer adjustment.

Key calculations performed every cycle:

- Individual cell voltages (Cell 1–4)
- Pack voltage (sum of all four cells)
- Average cell voltage
- Imbalance percentage: $((\text{highest} - \text{lowest}) / \text{average}) \times 100$
- Identification of the weakest and strongest cell

2.1 Health Classification Thresholds

Condition	Threshold	Health State
Lowest cell voltage	< 3.0 V	PACK FAILURE
Imbalance percentage	> 15%	CRITICAL IMBALANCE
Imbalance percentage	> 5%	MINOR IMBALANCE
All other conditions	—	HEALTHY

Each health state drives a dedicated LED (Green/Yellow/Red/Blue) and is reported in real time on the Serial Monitor and the LCD, forming the foundation that all subsequent tasks build upon.

3. Task 2 — Event-Driven Safety Protection Kernel

An automotive-grade, fully non-blocking protection controller was layered on top of Task 1. The entire kernel relies exclusively on **millis()** timestamps for all scheduling, with zero use of **delay()** anywhere in the runtime loop.

3.1 Fault Conditions Detected

Fault	Detection Logic
Weak Cell Voltage	Any cell < 3.0 V
Overvoltage	Any cell > 4.2 V
Sensor Anomaly	Reading < 2.0 V or > 4.5 V (outside physically valid range)

Fault	Detection Logic
Rapid Voltage Fluctuation	$ \text{current} - \text{previous reading} > 0.3 \text{ V between cycles}$

3.2 Protection Mechanisms

- **Relay Cutoff:** GPIO25-controlled relay (active LOW) disconnects the simulated load when a dangerous condition is detected.
- **Anti-Relay-Chatter Protection:** Relay state changes are gated by a minimum 2-second cooldown timer, preventing rapid on/off oscillation.
- **Non-blocking Buzzer Alerts:** The buzzer toggles every 500 ms using `millis()` comparisons rather than `tone()+delay()` blocking patterns.
- **Recovery Logic:** The system automatically re-evaluates conditions every cycle and restores the relay to NORMAL once all fault conditions clear, with the same anti-chatter cooldown applied to recovery.
- **Stable State Management:** A dedicated boolean state (*relayOpen*) is tracked and compared against the actual relay pin state every cycle, forming the basis of the Task 4 relay-mismatch fault check.

4. Task 3 — Intelligent Embedded HMI & Diagnostic Interface

A professional Human-Machine Interface was built on the 20x4 I2C LCD, automatically rotating through five diagnostic screens every 3 seconds. Screen transitions use `lcd.clear()` only at the moment of transition (not every refresh cycle), which together with millis()-gated refresh intervals, eliminates visible flicker.

Screen	Content Displayed
1 – Cell Voltages	Live voltage of all four cells (C1–C4)
2 – Pack Analytics	Pack voltage, average voltage, imbalance percentage
3 – Health Status	Current health state, weakest cell, strongest cell
4 – Protection	Relay status (NORMAL/CUT OFF), active warning message
5 – System / Cloud	Operational mode, WiFi status & RSSI, Blynk connection, queued telemetry count

A dedicated **Fault Priority Override** screen instantly preempts the normal rotation whenever a dangerous condition (Sensor Error, Overvoltage, Pack Failure, or Critical Imbalance) is detected, displaying the fault status, system mode, and relay state until the condition is resolved — satisfying the requirement for automatic fault-priority screen overrides during critical conditions.

5. Task 4 — Fault-Tolerant Embedded Runtime System

A fault-tolerant runtime subsystem autonomously detects hardware/software anomalies beyond simple threshold violations, and transitions the system through four distinct operational modes.

5.1 New Fault Detections Introduced

Fault Type	Detection Method
Sensor Disconnection	Raw ADC reading exactly 0 on any cell input
Frozen ADC	Cell voltage unchanged ($\Delta < 0.001$ V) for more than 5 consecutive read cycles
Relay Mismatch	Commanded relay state differs from the actual <code>digitalRead()</code> of the relay pin
Invalid Readings	Reused from Task 2's sensor anomaly check (outside 2.0–4.5 V)

5.2 Operational Mode State Machine

Mode	Trigger Condition
NORMAL	No faults; imbalance $\leq 5\%$; all sensors healthy
DEGRADED	Minor imbalance, fluctuation, or isolated single-cell fault present
FAILSAFE	Overvoltage, weak cell, or critical imbalance detected
SHUTDOWN	Sensor disconnected, relay mismatch, or 3+ simultaneous cell faults

Faults are isolated per cell using a *cellFault[4]* array, ensuring that a problem on one cell does not mask or incorrectly affect the diagnostic status of healthy cells. Every fault and mode transition is recorded in a circular *faultHistory[10]* buffer with a *millis()*-based timestamp, providing a structured, timestamped fault log that is printed to the Serial Monitor and surfaced to the cloud dashboard in Task 6.

6. Task 5 — Intelligent Cloud Telemetry Architecture

A smart IoT telemetry system was implemented using the Blynk Cloud platform over the ESP32's built-in WiFi (connected to the Wokwi-GUEST simulated network). Rather than transmitting on a fixed interval, telemetry is dispatched only when a meaningful change occurs — satisfying the event-driven requirement and minimising unnecessary cloud traffic.

6.1 Event-Driven Trigger Conditions

- Health state changes (e.g., HEALTHY → MINOR IMBALANCE)
- System operational mode changes (NORMAL → DEGRADED → FAILSAFE → SHUTDOWN)
- Relay status changes (NORMAL → CUT OFF or vice versa)
- Any active danger-state condition (anomaly, overvoltage, weak cell, critical imbalance)

6.2 Resilience Features

- **WiFi Reconnect Handling:** Connection status is polled every 5 seconds; on disconnection, `WiFi.begin()` is automatically re-invoked.
- **Event Queue Synchronization:** Up to 10 telemetry events are buffered locally in a circular queue while offline, then flushed to Blynk in order immediately upon reconnection.
- **Signal Quality Monitoring:** WiFi RSSI is read and displayed on the LCD's System/Cloud screen as a live signal-strength indicator.
- **Asynchronous, Non-blocking Communication:** `Blynk.run()` is called inside a `millis()`-gated 100 ms interval, never blocking the main sensing or protection loop.
- **Continued Local Operation:** All protection, LED, buzzer, relay, and LCD functions continue to operate normally and immediately even when WiFi/Blynk is fully disconnected.

7. Task 6 — Executive Battery Intelligence Dashboard

An enterprise-grade Blynk web dashboard was built on top of the Task 5 telemetry pipeline, providing a structured, professional operator view of the entire battery pack.

7.1 Dashboard Widgets & Datastreams

Virtual Pin	Widget	Purpose
V0–V3	Gauge (severity colour-coded)	Individual cell voltages (Cell 1–4)
V4	Gauge	Pack voltage (0–20 V range)
V5	Gauge	Average cell voltage

Virtual Pin	Widget	Purpose
V6	Gauge	Imbalance percentage (inverted colour logic — high = red)
V7	Value Display	Live health status text
V8	Value Display	Live operational mode text
V9	Value Display	Live relay status text
V10	Value Display	Most recent fault from the on-device fault log, with timestamp
V11	Value Display	Auto-generated operator recommendation
—	Custom Chart	Historical cell-voltage trend (V0–V3) over time

7.2 Severity-Based Colour Coding

All voltage gauges use a three-zone colour gradient: **Red** for dangerous/low voltages, **Yellow** for warning-range voltages, and **Green** for safe, healthy voltages. The Imbalance gauge uses the inverse mapping (green at low imbalance, red at high imbalance), ensuring the colour always represents true risk severity rather than raw numeric value.

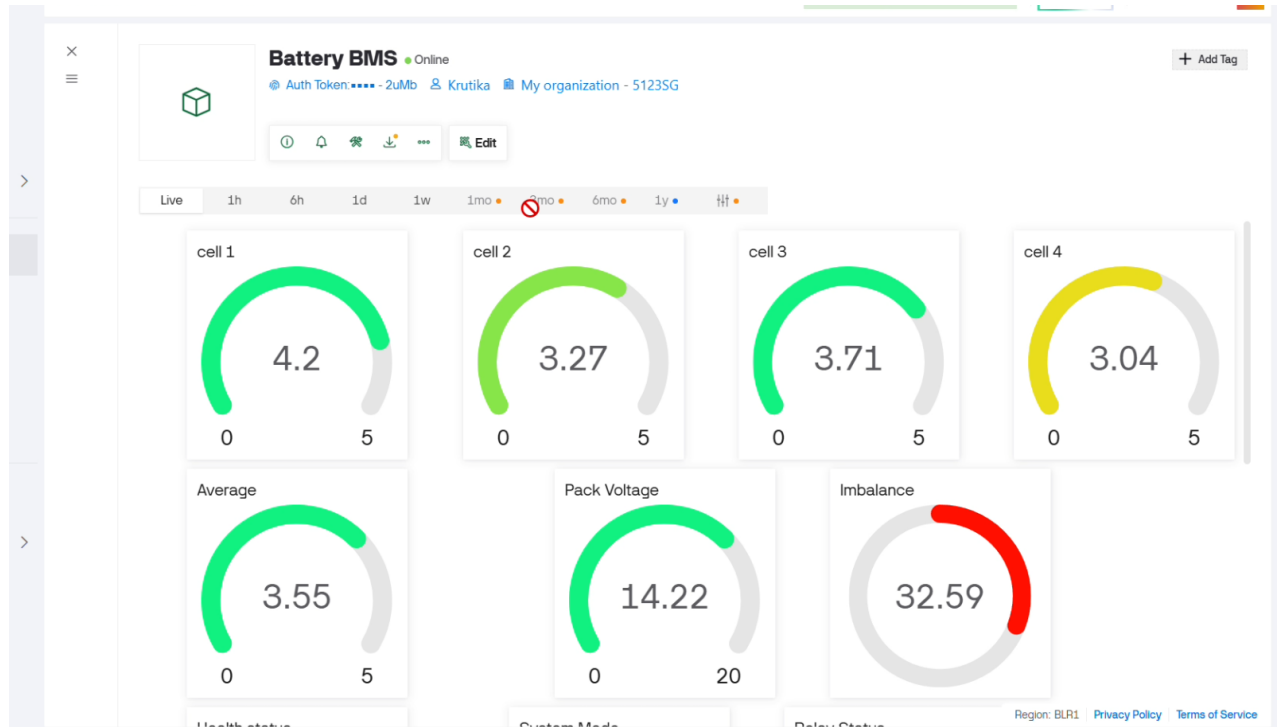
7.3 Intelligent Operator Recommendations

A rule-based recommendation engine maps the current health state directly to a concise, actionable instruction for the operator (e.g., “Stop use! Replace weak cell.” for PACK FAILURE, or “System healthy. No action needed.” when HEALTHY), delivering the actionable-insight requirement of a professional IoT user experience.

8. Live Dashboard Evidence

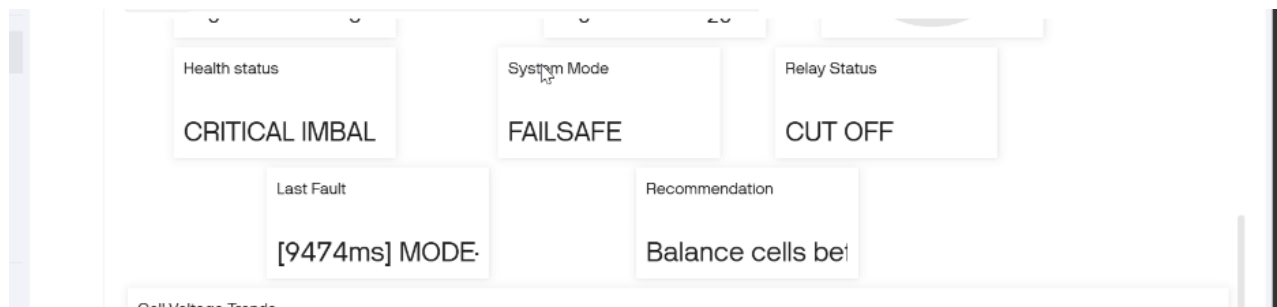
The screenshots below were captured directly from the live Blynk Web Dashboard (blynk.cloud) while the Wokwi simulation was actively running, confirming that the device successfully connects to the cloud, transmits telemetry, and renders all configured widgets in real time.

Cell Voltage & Pack Analytics Gauges



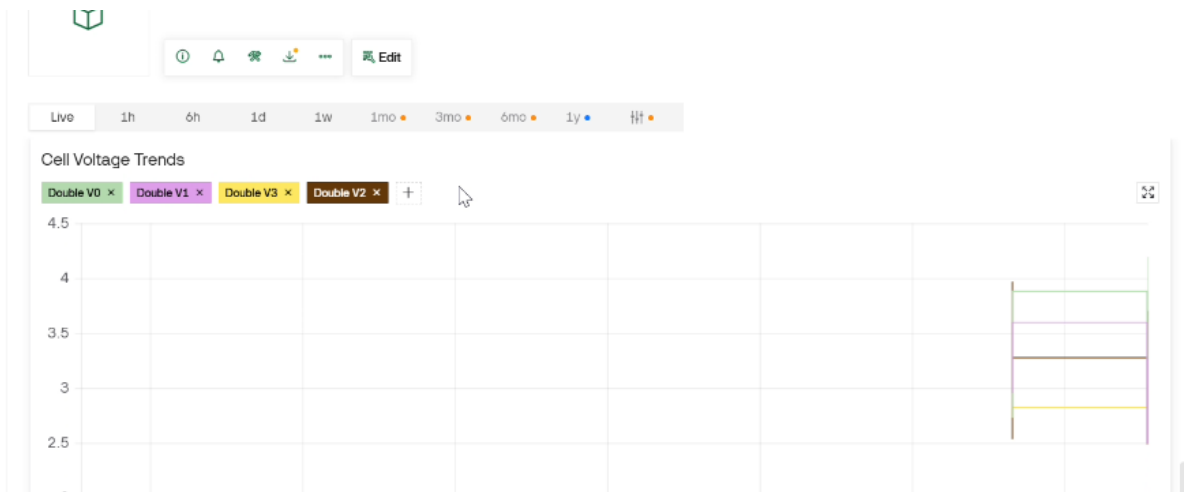
Live gauges for Cell 1–4, Average, Pack Voltage, and Imbalance, each coloured according to severity (green = safe, yellow = caution, red = high risk). At the moment of capture, Cell 4 was reading 3.04 V (yellow zone) and Imbalance was 32.59% (red zone), correctly reflecting a CRITICAL IMBALANCE condition.

Health Status, System Mode, Relay Status, Fault Log & Recommendation



The dashboard correctly reports Health Status as *CRITICAL IMBAL*, System Mode as *FAILSAFE*, and Relay Status as *CUT OFF* — consistent with the gauge readings above. The Last Fault widget shows a timestamped mode-transition log entry, and the Recommendation widget displays the corresponding operator action (“Balance cells before next use.”), confirming Task 4, 5, and 6 are all working together correctly.

Cell Voltage Trends (Historical Chart)



The Custom Chart widget plots historical cell voltages (V0–V3) over time using 1-minute average aggregation, providing the operator with a visual trend of pack behaviour rather than only the latest snapshot.

9. System Integration

All six tasks share a single global data model (cell voltages, pack statistics, health state, operational mode, fault log, and connectivity status) and run inside one unified Wokwi project, one circuit diagram, and one continuous firmware loop — with no separate sketches, boards, or simulations per task. The data flow is summarised below:

Stage	Description
1. Sensing (Task 1)	Read 4 potentiometers → map to lithium voltage range → compute pack statistics
2. Protection (Task 2)	Evaluate thresholds → drive LEDs/buzzer → control relay with anti-chatter logic
3. Diagnostics (Task 4)	Detect hardware/software faults → isolate per cell → update operational mode → log fault history
4. Display (Task 3)	Rotate through 5 LCD screens → instantly override to fault screen when dangerous
5. Telemetry (Task 5)	On state change, push data to Blynk Cloud (or queue if offline) → auto-reconnect WiFi/Blynk

Stage	Description
6. Dashboard (Task 6)	Blynk widgets render live gauges, status labels, fault history, recommendations, and trend chart

This single-loop, event-driven architecture ensures that sensing, protection, diagnostics, display, and cloud communication all execute concurrently without blocking one another, fulfilling the scalable and modular embedded architecture principle requested across the task set.

10. Testing & Observed Results

- All four health states (HEALTHY, MINOR IMBALANCE, CRITICAL IMBALANCE, PACK FAILURE) were successfully triggered by adjusting potentiometer positions and verified on the LCD and Serial Monitor.
- The relay correctly cut off (active LOW, GPIO25) during PACK FAILURE and CRITICAL IMBALANCE conditions, with the 2-second anti-chatter cooldown preventing rapid toggling.
- The fault-priority LCD screen correctly overrode the normal 5-screen rotation immediately upon fault detection and returned to normal rotation upon recovery.
- Operational mode transitions (NORMAL → DEGRADED → FAILSAFE → SHUTDOWN) were confirmed in the Serial Monitor fault log with correct millis() timestamps.
- Blynk Cloud connectivity was established successfully (WiFi via Wokwi-GUEST, followed by an explicit Blynk.connect() call), with the device showing Online status on the Blynk console.
- Event-driven telemetry was confirmed: dashboard values updated only on health/mode/relay state changes, consistent with the Task 5 specification, rather than on a fixed polling interval.
- All Blynk gauges were configured with severity-based colour gradients (red/yellow/green), and the Custom Chart widget displayed historical voltage trends using 1-minute average data aggregation.

11. Conclusion

This project successfully delivers a complete, end-to-end Battery Management System spanning sensing, real-time protection, diagnostic HMI, fault-tolerant runtime management, cloud telemetry, and an executive-level monitoring dashboard — all integrated into a single unified Wokwi simulation as required. The implementation demonstrates scalable, modular, non-blocking embedded architecture principles throughout, and the resulting system behaves consistently with real-world automotive-grade BMS design practices.

Project: Adaptive Multi-Cell Battery Intelligence Engine (Unified BMS)
Live Wokwi Project: <https://wokwi.com/projects/467059005329336321>